

Software Project 2 Specification: Stock Data Chart Viewer

Windows Forms Application for Stock Data Analysis

Project Overview

Project 2 will be a slight, but meaningful, modification of project 1. In project 2, you will now allow the user to select and display more than 1 stock at a time. Unlike project 1, there will be **no DataGridView**. Instead, the display of data will be in the form of a Candlestick chart. Your project should display the first selected stock in the main window and then display the other stocks in their own individual forms, as we showed in class.

Requirements

- **Compilation:** Your code MUST compile without errors. If your code doesn't compile, your project will receive a grade of 0.
- **Running:** Obviously, your code must run. A project that doesn't run will receive a grade of 0.
- **Easy Start:** The loading button must have TabIndex 0 so that it has the focus when the application starts. Similarly, the starting and ending date should be January 28, 2021 to February 28, 2021. The OpenFileDialog should also be preset to open the "ABBV-daily.csv" file. When your application is started, the main form must immediately show the ABBV_Daily.csv stock between the dates above, before the user selects any other stocks.
- **Naming Convention:** Make sure you name your controls in the way we specified it in class. You are also to use Camel case, For example the button to load a stock should not be called Button1 (as Visual Studio would) but as Button_SomeMeaningfulName (e.g. Button_LoadStock)
- **File naming:** If you are creating new classes, make sure that the name of the file associated with the new class matches the name of the class. This makes it easy to find where a class is defined/implemented.
- **Comments:** Make sure your code is well commented. Functions headers should be preceded with a "///" comment so that the function's usage is automatically

displayed. Inside the function implementation should have “//” type comments inside the function to explain the working of the function.

Application Features

1. **Initial Display:** AGAIN, when your application is run, the main form should show the ABBV_Daily.csv stock even before the user selects any other stocks. Think how the standard application starts by using the default constructor of the main form. That is why in project #1, the main form was initially empty until the user selected a stock. Now, the application in project #2 should start with a display of the ABBV_Daily.csv stock with the initial start and end dates specified in the “Easy Start” paragraph. After that, the user can click the load button and select multiple stocks from the list of available CSV files from the *Stock Data* folder. Users should be able to select any number of files representing different stocks.
2. **Stock Selection:** : The application must display a list of available CSV files from the *Stock Data* folder. Users should be able to select any number of files representing different stocks.
3. **Data Display:** Upon selection, the application should read the chosen CSV files and display the data in the Chart control of each individual form.
4. **Date Range Input:** Users must be able to specify a start date and end date to view only the stock data within that range.
5. **Update Functionality:** An *Update* button must be provided. When pressed, the Chart should be refreshed to show data according to the selected date range.
6. **Date Range Modification:** Users should have the option to change the date range and update the display as often as needed in any of the forms.
7. **Period Specification:** The user must be able to display all the stock files or just the daily, just the weekly, monthly or yearly periods.

Technical Specifications

- **Framework:** NET Framework or .NET Core
- **UI Framework:** Windows Forms
- **Controls:** utilize the controls we used in class.

- **Data Handling:** Use appropriate objects from the .NET framework libraries to read and parse CSV files. DO NOT USE outside classes like CSVHelper. You are to write your own data loading class (as we did in class).

Submission Instructions

- Submit the complete Visual Studio solution by CLEANING (Clean in the menu) and then ZIPPING your solution folder (NOT the project folder which is inside the solution folder). Do not include the “Stock Data” file. That file should be in the parent folder of your solution folder.
- A rubric will be provided shortly.
- I have tried to keep instructions simple to minimize confusion. But feel free to ask questions if you have any.
- You can use the suggestions your instructor makes in class, or do your own implementation, or even use AI to help you design your project. HOWEVER, you want to make sure you fully understand the code you submit. Your level of comprehension of your project will be assessed via a quiz, as was done after project 1.
- You are STRICTLY FORBIDDEN from getting your project from GitHub and submitting it as your own. That would be considered CHEATING and would subject your project to an immediate 0 and your overall grade would be an F,
- Make SURE you understand the specifications. If you need ANY CLARIFICATIONS, ask for them in class!!!

Final instruction regarding the use of AI. You have been told that the use of AI is permitted. However, you need to make sure that you TRULY understand the code provided to you. In order to simplify things, you are not to use the code generated by AI to control the Chart control. The only way you are allowed to control the Chart is via the IDE by setting some of its properties (ChartArea, Series, etc...) as we did in class!!! The only allowed approach for displaying the stock data in the chart is by setting the Chart’s DataSource and calling its DataBind() method! The last 2 requirements are implemented via 2 lines of code!!!

Even the normalization of the chart can be done via a property of the chart. A chart can be normalized by setting The chart -> AxisY -> IsStartedFromZero = false. In other words, you simply ask a particular chart area to have their Y axis NOT START from 0.

So, if it is found that you are using AI code to control the chart, you will receive a 0 on your project.

This can be done in the properties editor in the IDE. So, again, setting a single property will perform the normalization for you!!! You no longer need to write the normalization code (though it is a simple method).